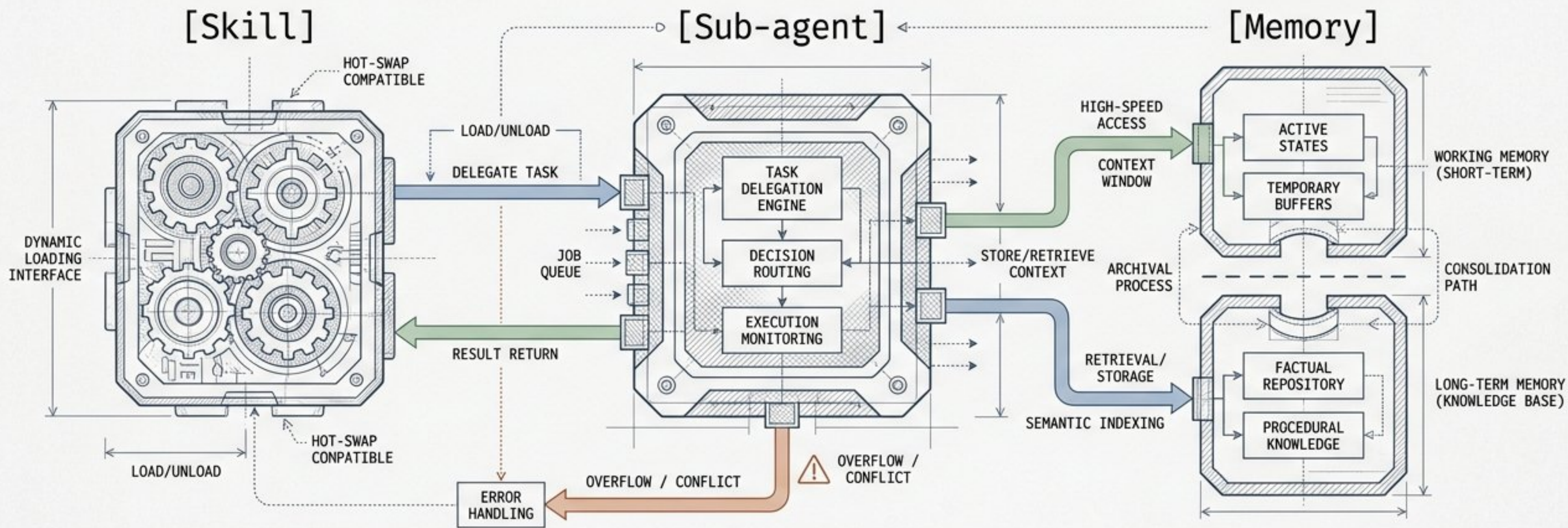


Agent 能力装配系统设计

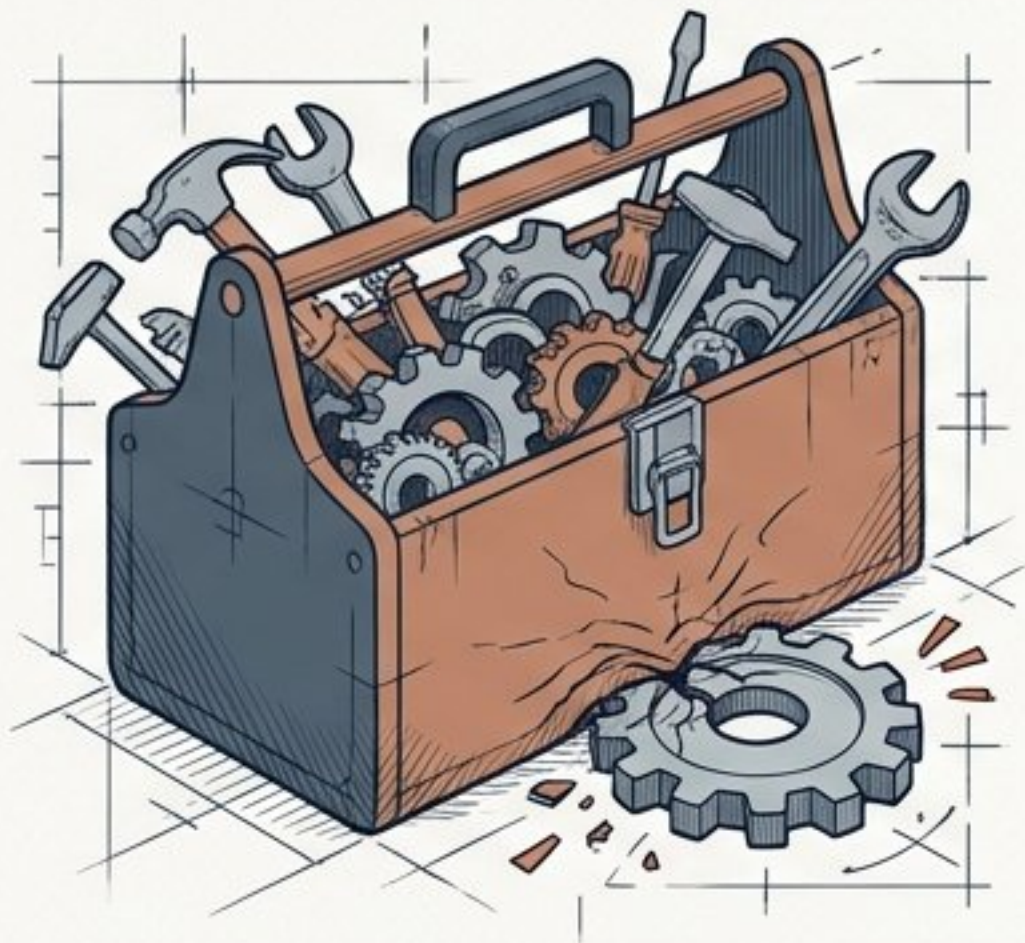
Skill 动态加载、Sub-agent 委托与双层 Memory 架构



致命的工程矛盾：能力扩张 vs. 上下文坍塌

Agent 只有一套能力时，必然陷入“工具太多上下文爆炸，太少做不了事”的死局。

Everything-in-System-Prompt



20 种能力 × 平均 500 Token = 10,000 Token 仅用于指令。
后果：指令稀释（如同在 200 页手册中找一句话），延迟极高。

上下文无限膨胀



复杂任务循环 20 轮 = 累积 20,000+ Token 历史记录。
后果：触发 Lost in the Middle 效应，过程细节淹没核心目标。

> System Error: 单一 Agent Loop 无法应对复杂任务，必须进行架构级“信息压缩”。

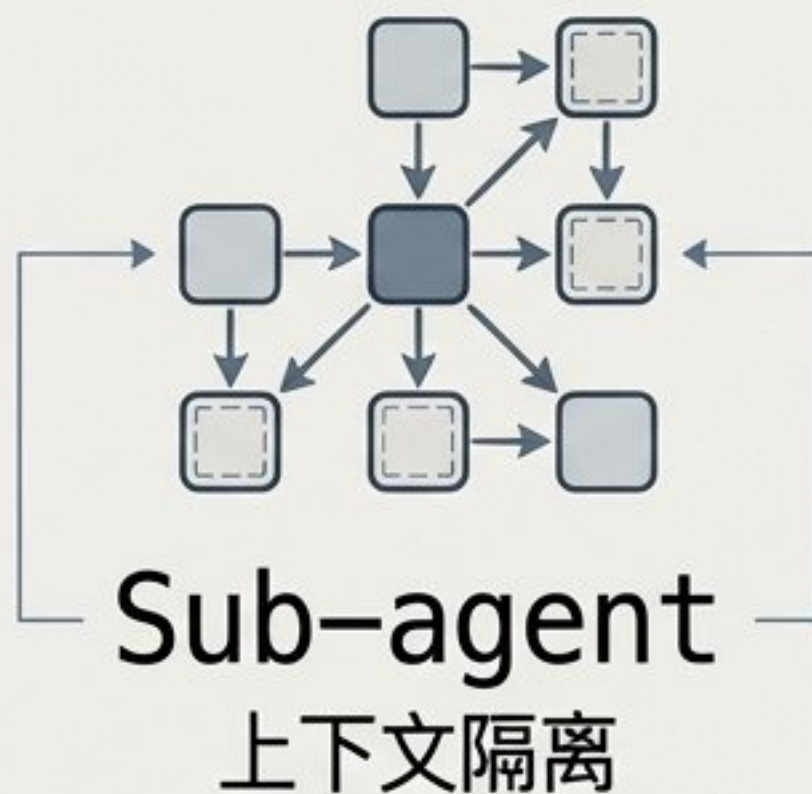
破局之道：三大核心机制与 OS 架构映射

将 Agent 视作“计算机操作系统”进行按需装配，在有限窗口内处理无限复杂度的任务。



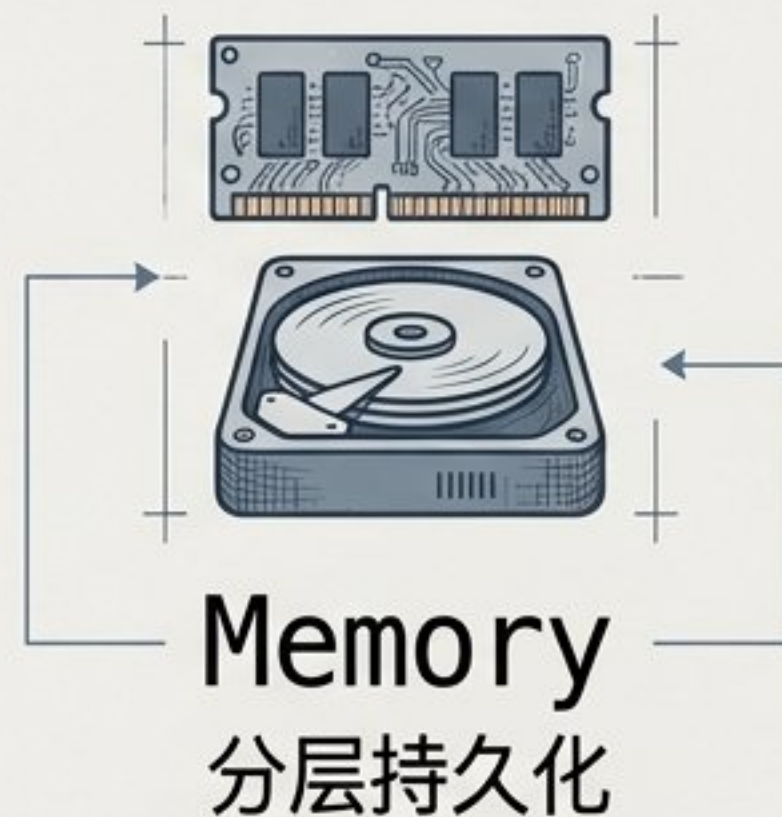
OS 隐喻：动态链接库 (DLL / Plugins)

- **机制**：结构化的 System Prompt 片段，按需动态注入。
- **解决**：指令稀释与初始上下文爆炸。



OS 隐喻：隔离的子进程 (Child Processes)

- **机制**：将复杂子任务委托至独立上下文，仅回收最终结果。
- **解决**：过程日志导致的历史膨胀与干扰。



OS 隐喻：多级缓存体系 (RAM & SSD)

- **机制**：短期缓冲区摘要压缩 + 长期向量库持久化。
- **解决**：记忆遗忘与无限增长。

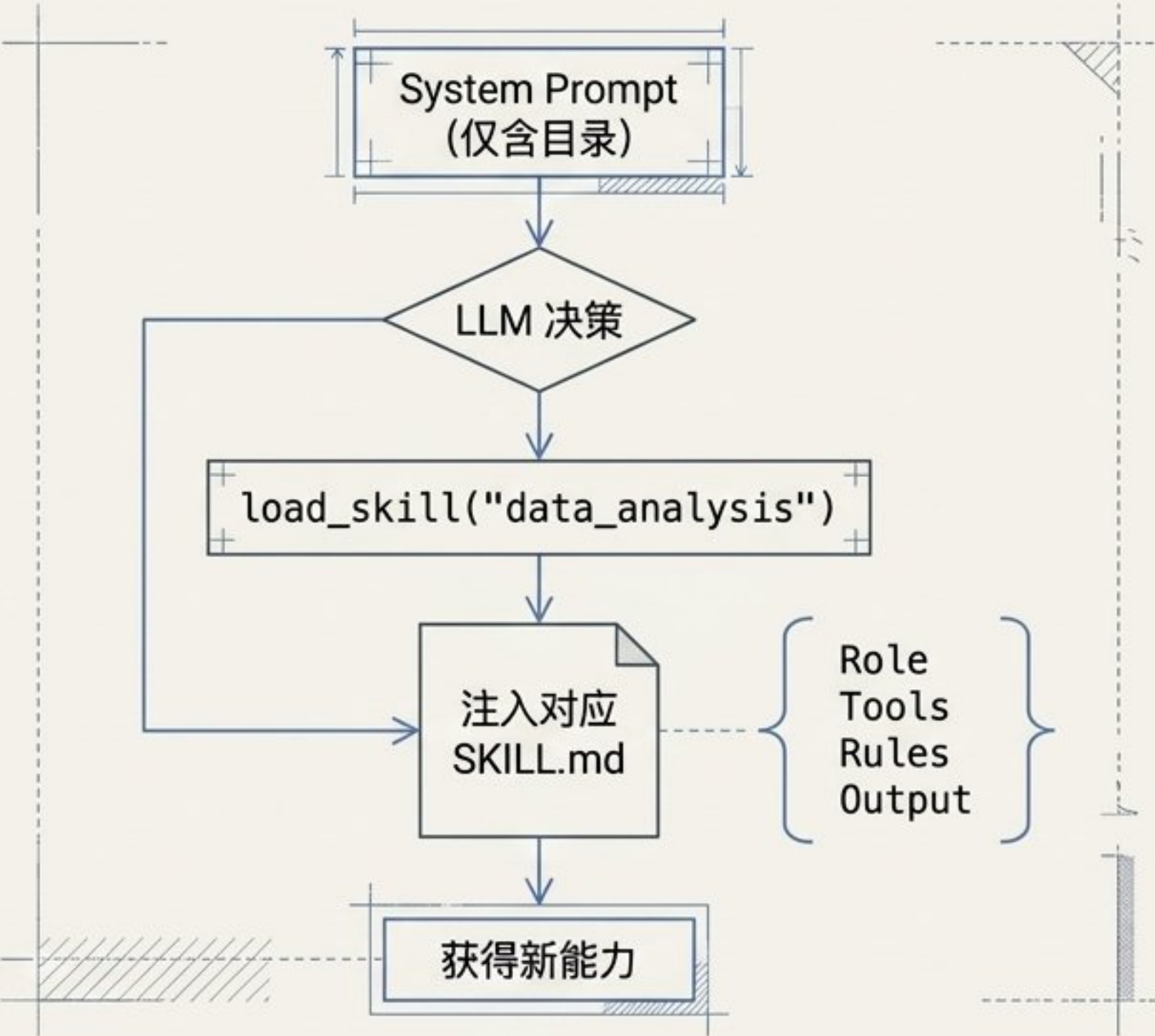
Pillar 1: Skill 模式与渐进式呈现 (Progressive Disclosure)

Comparison Matrix

	Everything-in-System-Prompt
	零延迟，但极度冗余。 隐喻：带着所有工具箱爬山。
	Progressive Disclosure 按需加载，上下文精简， 需额外路由步骤。 隐喻：带有分类抽屉的工作台。

核心洞察：用工具来管理工具——把 Skill 的选择变成一种 Function Calling 操作。

Topology Flowchart - The Meta-tool Pattern



[深度剖析] Skill 的代数本质与延迟求值

Model Insight:
Claude Opus 4.7
(2026-05-05)

百种技能同时加载，系统崩溃

全集加载代价: $\sum |S_i|$

索引加载代价: $N \cdot \text{const} + |S_{\text{loaded}}|$

目录压缩表示 + 运行时展开

Code Annotation

```
1 // The Information Theory Perspective:
2 好的 Skill 目录是‘压缩表示’，让 LLM
3 实现 Lazy Evaluation (延迟求值)。在
4 目录层就能做出正确 Routing，无需展开
5 全文。
6
7 // Engineering ROI:
8 Skill description 的工程投入回报远高于 Skill 逻辑本身。描述模糊导致路由错误，直接引发任务失败。这是 SKILL.md 成为事实标准的根本原因。
```


Pillar 2: Sub-agent 与信息瓶颈机制

常见误区：Sub-agent 不是“多个 AI 在群聊”。正确实践：“委托 - 执行 - 返回结果”的单线隔离关系。

主 Agent 输入：下发任务指令

~500 tokens

隐喻：CEO 派发“设计 Logo”任务。

Sub-agent 内部黑盒：思考与试错

~15,000 tokens

返回主 Agent：仅输出最终结果 ~500 tokens

隐喻：设计团队直接交付 Logo，CEO 不听会议录音。

核心结论：Sub-agent 的本质是利用上下文隔离（Context Isolation）实现物理级的信息压缩。

拆分 Sub-agent 的工程决策与 ROI 模型

拆分子智能体有极高的上下文和调用开销。简单任务盲目拆分会导致负收益。

ROI Decision Matrix Data Table				
任务类型	内部 Token	返回 Token	压缩比	决策
修 Bug (Debugging)	~15K	~500	30x	[强烈建议拆分]
搜索 + 总结十篇文档	~20K	~2K	10x	[建议拆分]
写 100 行代码	~5K	~3K	~2x	[视情况而定]
简单计算 (1+1)	~200	~10	20x (绝对值低)	[负收益 - 开销>收益]

Sub-agent 触发拆分检查清单 (Checklist)

- ☐ 1. 需完全不同上下文?
- ☐ 2. 需 5 步以上循环?
- ☐ 3. 需失败隔离?

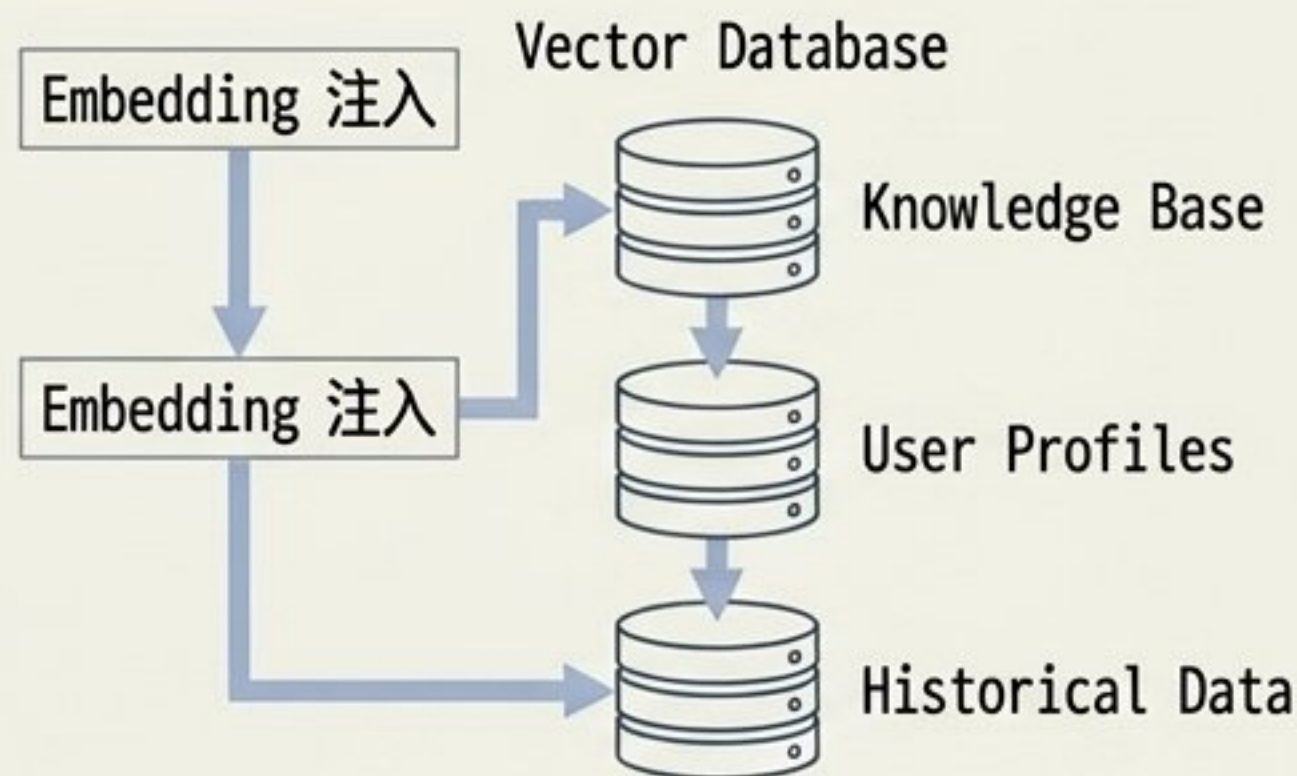
Pillar 3: Agent 的双层记忆持久化架构

Short-term Memory (RAM / 缓冲区)



- 存储内容: 当前对话与工具结果 (Current Dialogue & Tool Results)
- 生命周期: 对话结束即丢弃 (Discarded at session end)
- 限制: 受限于 Context Window (Limited by Context Window)

Long-term Memory (SSD / 外部存储)



- 存储内容: 用户偏好、跨对话经验 (User Preferences & Cross-Session Experience)
- 生命周期: 持久化可用 (Persistently Available)
- 限制: 理论容量无限 (Theoretically Unlimited Capacity)



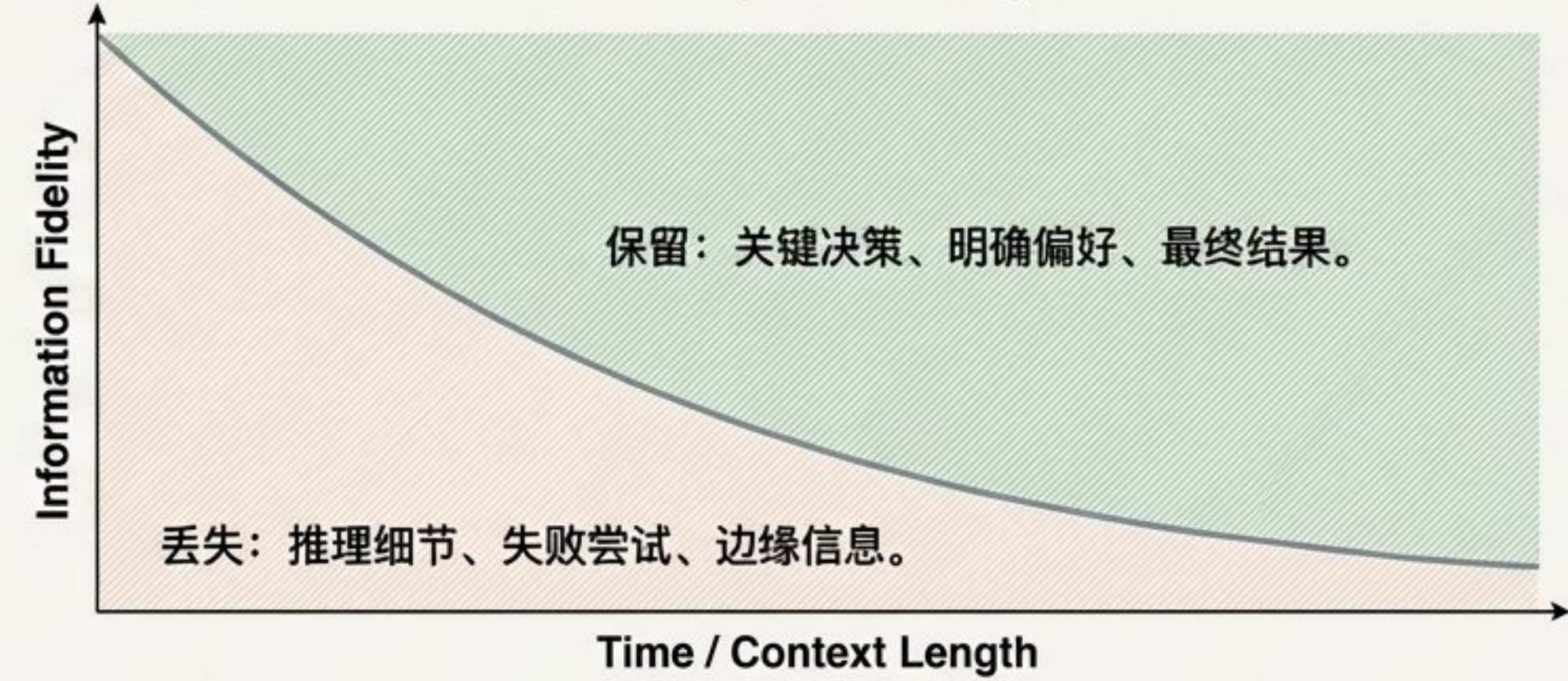
工程实践标注: 用户第 1 次用 15 步生成图表, 系统提取‘偏好蓝色系/Markdown格式’注入长期记忆。第 2 次仅需 8 步即完成。

[深度剖析] OS 内存层级对偶与上下文压缩代价

OS Hierarchy Mapping Table

	OS Concept	LLM Concept
1	L1/L2 Cache	LLM KV Cache (最低延迟, 自动管理)
2	RAM	Context Window (短期记忆, 热数据)
3	SSD	Vector DB (长期记忆, 冷/温数据)

Context Compaction Decay Curve



System.Warning(LLM-as-summarizer Bias):

压缩是由另一个 LLM 完成的, 若它系统性地忽视某类细节 (如代码边界条件), Agent 将陷入“重复犯同类错误”的死循环。结论: 能不压就不压, 将压缩作为 80% 容量时的最后手段。

长期记忆的隐性代价：隐私诉讼的种子

长期累积让 Agent 对用户的“隐式画像”可能比用户愿意分享的更精确。
没有隐私设计的记忆系统在生产环境中是不合格的。

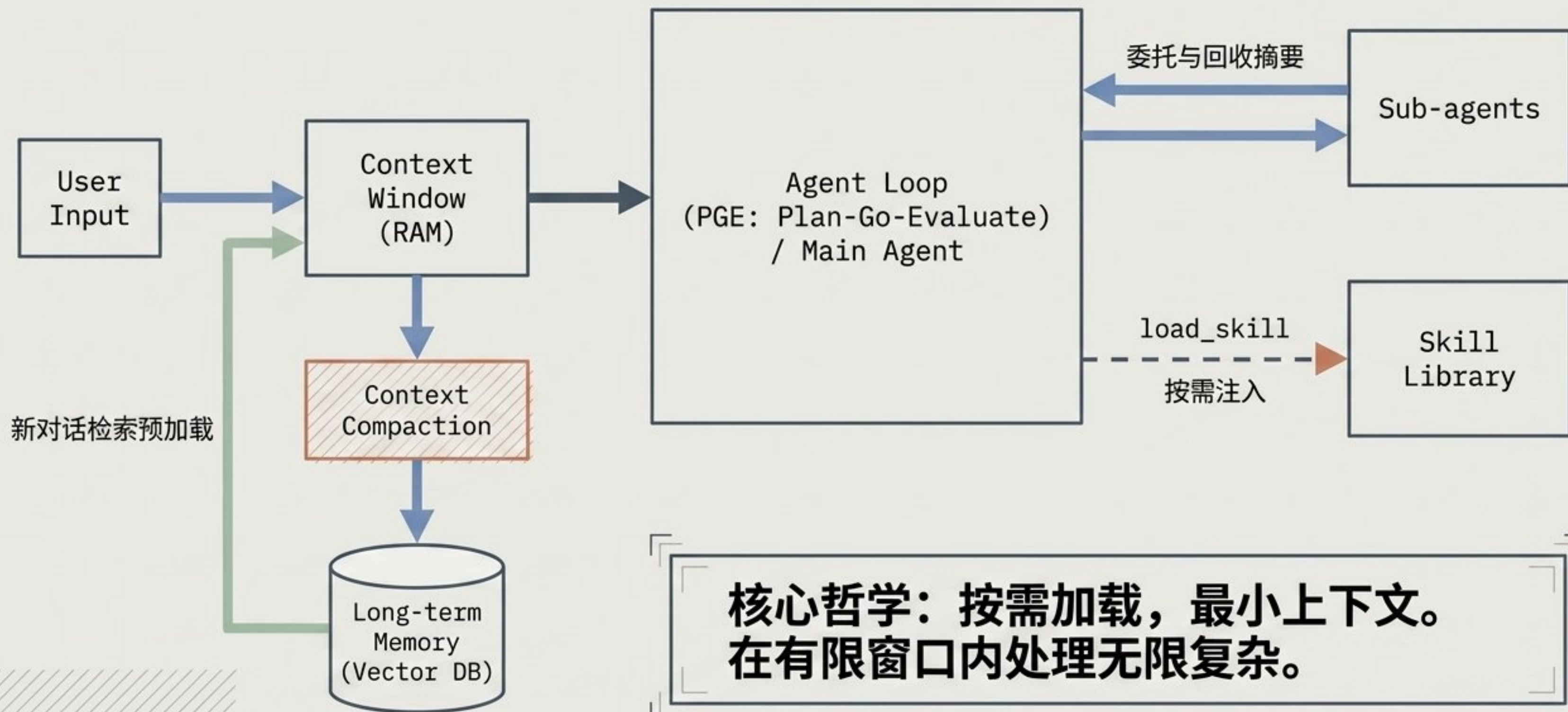
红色警戒区

- | | |
|---|-------------------------------|
| ✗ | 跨会话泄露 (Cross-session Leakage) |
| ✗ | 隐式画像越界 (Implicit Profiling) |
| ✗ | 记忆投毒攻击 (Memory Poisoning) |
| | |

绿色防御墙

- | | |
|---|---|
| ✓ | 严格的租户级用户隔离
(Strict User Isolation) |
| ✓ | 强加密标准 (At-rest + In-transit) |
| ✓ | 可删除性与 GDPR 合规
(Right to be Forgotten) |
| ✓ | 用户可审计性
(Transparent Memory Management) |

终极重构：完整 AI 助手系统蓝图



认清边界：生产环境排雷指南与反模式

过度工程 (Over-engineering)

把极简任务硬包成 Skill，不仅增加 Prompt 长度反提升失败率。数据：Voyager 论文中 30%+ 的 Skill 从未被复用。

泛化伪证 (Domain Fallacy)

跨领域 Skill 复用的有效性极度脆弱。不要迷信实验室里的论文数据（如 Minecraft/WebArena），无法直接套用到企业级业务。

版本污染 (Version Pollution)

自动生成的 Skill 带有 Bug，被频繁调用导致错误指数级传播。系统必须建立严格的评分与自动退役机制。

混淆 Skill 与微调 (FT vs. In-context)

Skill 解决的是“高频变化”的逻辑路由；微调 (Fine-tuning) 改变的是“底层能力分布”。混用将导致灾难性结果。

2026 年 Skill 生态现状速览

按需装配架构已彻底从实验室走向商业基座事实标准。



趋势预判: Skill Marketplace 将成为下一个 App Store; 而基于信息瓶颈的 Sub-agent 已成为所有顶级生产环境代码助手的标配。

从 Prompt 技巧到精密的信息架构设计

- 1 Skill: 解决能力激增的目录索引与动态按需加载。
- 2 Progressive Disclosure: 工程回报率最高的设计哲学。
- 3 Sub-agent: 通过信息瓶颈实现纯物理级的上下文隔离。
- 4 Memory: 双层映射 (RAM + 向量检索) 解决遗忘与爆炸。
- 5 PGE + 3 Pillars: 拼图完成, 真正的 AI 助手诞生。

当你的 Agent 需要调用另一家公司的 CRM, 或者与外部组织的 Agent 协作时怎么办?



预告: L28: Agent Protocols

从 MCP 到 Agent-to-Agent 通信协议, 跨越组织边界的握手。